

基于ChatGPT辅助优化的实用JS工具方法集

在前端开发中，重复的业务逻辑和通用功能处理会降低开发效率。借助ChatGPT的逻辑拆解与边界场景预判能力，我们可以开发出覆盖多场景、容错性强的JS工具方法。本文整理了10个高频实用的工具方法，包含数据校验、格式转换、数组处理、DOM操作等核心领域，每个方法均经过异常处理优化和性能考量。

一、工具方法设计原则（ChatGPT辅助提炼）

在ChatGPT的逻辑梳理下，实用JS工具方法需遵循以下原则，确保可用性与扩展性：

- 单一职责**：每个方法仅处理一个核心功能，便于复用与维护；
- 边界容错**：针对 `null`、`undefined`、异常数据类型等边界场景做兼容处理；
- 参数明确**：通过JSDoc清晰定义参数类型、含义及默认值；
- 返回一致**：相同场景下返回格式保持统一（如布尔值校验统一返回true/false）；
- 性能优化**：避免不必要的循环与DOM操作，减少性能损耗。

二、核心工具方法实现（附ChatGPT优化思路）

以下工具方法按使用频率排序，每个方法均包含“功能描述”“实现代码”“ChatGPT优化点”三部分，清晰呈现优化逻辑与价值。

1. 数据类型精准判断（替代typeof的缺陷）

功能描述：解决typeof无法区分数组、null、对象等类型的问题，返回精准的数据类型字符串（如"array"、"null"、"date"）。

Code block

```
1  /**
2   * 精准判断数据类型
3   * @param {*} value - 待判断的值
4   * @returns {string} 数据类型字符串（小写，如"string"、"array"、"null"）
5   */
6  export const getDataType = (value) => {
7    // 利用Object.prototype.toString获取原生类型标签
8    const typeTag = Object.prototype.toString.call(value);
9    // 匹配"[object Xxx]"中的Xxx部分，并转为小写
10   const type = typeTag.match(/\[object (\w+)\]/)[1].toLowerCase();
11   return type;
12  };
```

```
13
14 // 使用示例
15 getDataType("hello"); // "string"
16 getDataType([]); // "array"
17 getDataType(null); // "null"
18 getDataType(new Date()); // "date"
19 getDataType({}); // "object"
```

ChatGPT优化点：

- 摒弃typeof的模糊判断，采用Object.prototype.toString这一原生可靠方案，覆盖null、数组、日期等特殊类型；
- 通过正则提取类型标签，避免手动截取字符串导致的容错问题；
- 统一返回小写字符串，与typeof返回格式保持一致，降低使用成本。

2. 深拷贝（支持复杂数据类型）

功能描述：实现对象/数组的深拷贝，支持循环引用、日期、正则等特殊类型，解决JSON.parse(JSON.stringify())的局限性。

Code block

```
1  /**
2   * 深拷贝数据（支持复杂类型与循环引用）
3   * @param {*} target - 待拷贝的目标数据
4   * @param {WeakMap} [cache=new WeakMap()] - 缓存容器，用于处理循环引用（外部无需传递）
5   * @returns {*} 拷贝后的新数据
6   */
7  export const deepClone = (target, cache = new WeakMap()) => {
8    // 1. 基础类型与函数直接返回（函数拷贝意义不大，直接复用引用）
9    const basicTypes = ["string", "number", "boolean", "undefined", "symbol",
10     "bigint"];
11    const targetType = getDataType(target);
12    if (basicTypes.includes(targetType) || targetType === "function") {
13      return target;
14    }
15    // 2. 处理循环引用：若缓存中存在，直接返回缓存的拷贝对象
16    if (cache.has(target)) {
17      return cache.get(target);
18    }
19
20    // 3. 特殊类型处理
21    let cloneResult;
22    if (targetType === "date") {
```

```

23     cloneResult = new Date(target.getTime());
24 } else if (targetType === "regexp") {
25     // 拷贝正则的源码、全局标志、忽略大小写等属性
26     cloneResult = new RegExp(target.source, target.flags);
27 } else if (targetType === "array") {
28     cloneResult = [];
29 } else if (targetType === "object") {
30     cloneResult = {};
31 }
32
33 // 4. 将当前对象存入缓存，避免循环引用
34 cache.set(target, cloneResult);
35
36 // 5. 递归拷贝属性（数组用forEach，对象用Object.keys）
37 if (targetType === "array") {
38     target.forEach((item, index) => {
39         cloneResult[index] = deepClone(item, cache);
40     });
41 } else if (targetType === "object") {
42     Object.keys(target).forEach((key) => {
43         cloneResult[key] = deepClone(target[key], cache);
44     });
45 }
46
47 return cloneResult;
48 };
49
50 // 使用示例
51 const obj = { name: "张三", info: { age: 20 }, arr: [1, 2, 3] };
52 obj.self = obj; // 循环引用
53 const cloneObj = deepClone(obj);
54 cloneObj.info.age = 25;
55 console.log(obj.info.age); // 20 (原对象不受影响)
56 console.log(cloneObj.self === cloneObj); // true (循环引用处理正常)

```

ChatGPT优化点：

- 引入WeakMap作为缓存容器，既解决循环引用问题，又能让缓存对象随目标对象被垃圾回收，避免内存泄漏；
- 单独处理日期、正则等特殊类型，弥补JSON拷贝无法保留这些类型的缺陷；
- 基础类型与函数直接返回，减少不必要的递归，提升性能；
- 复用前文的getDataType方法，保持工具集的一致性。

3. 防抖（Debounce）

功能描述：限制函数在指定时间内只能执行一次，常用于输入框搜索、窗口resize等高频触发场景。

Code block

```
1  /**
2   * 函数防抖
3   * @param {Function} func - 待防抖的函数
4   * @param {number} delay - 延迟时间 (毫秒)
5   * @param {boolean} [immediate=false] - 是否立即执行 (true: 先执行后防抖, false: 先
   防抖后执行)
6   * @returns {Function} 防抖后的函数
7   */
8  export const debounce = (func, delay, immediate = false) => {
9    let timer = null; // 定时器标识
10
11    // 返回防抖函数, 保留this指向与函数参数
12    return function (...args) {
13      const context = this;
14
15      // 清除之前的定时器
16      if (timer) clearTimeout(timer);
17
18      // 立即执行逻辑
19      if (immediate) {
20        // 若定时器不存在, 说明当前是第一次触发或已超过延迟时间, 执行函数
21        const isExecute = !timer;
22        // 重置定时器, 确保延迟时间内不再执行
23        timer = setTimeout(() => {
24          timer = null;
25        }, delay);
26        if (isExecute) func.apply(context, args);
27      } else {
28        // 非立即执行: 延迟后执行函数
29        timer = setTimeout(() => {
30          func.apply(context, args);
31          timer = null;
32        }, delay);
33      }
34    };
35  };
36
37  // 使用示例
38  const search = debounce((keyword) => {
39    console.log("搜索: ", keyword);
40  }, 500);
41  // 输入框输入时调用, 500ms内连续输入不会重复触发
42  input.addEventListener("input", (e) => search(e.target.value));
```

ChatGPT优化点：

- 添加immediate参数，支持“立即执行”模式（如搜索框首次输入立即触发，后续输入防抖），满足更多场景；
- 通过apply绑定this指向，确保函数内部this指向正确（如Vue组件中使用时指向组件实例）；
- 接收...args参数，保留原函数的参数传递能力；
- 定时器执行后重置为null，避免内存占用，同时辅助immediate模式判断。

4. 节流 (Throttle)

功能描述：限制函数在指定时间内只能执行一次，与防抖的区别是：节流确保固定时间内必然执行一次，常用于滚动加载、按钮防重复点击。

Code block

```
1  /**
2   * 函数节流
3   * @param {Function} func - 待节流的函数
4   * @param {number} interval - 节流间隔 (毫秒)
5   * @param {Object} [options={ leading: true, trailing: true }] - 配置项: leading
   (是否执行第一次)、trailing (是否执行最后一次)
6   * @returns {Function} 节流后的函数
7   */
8  export const throttle = (func, interval, options = { leading: true, trailing:
true }) => {
9    const { leading = true, trailing = true } = options;
10    let lastTime = 0; // 上一次执行时间
11    let timer = null; // 定时器标识 (用于处理trailing)
12
13    return function (...args) {
14      const context = this;
15      const nowTime = Date.now();
16
17      // 1. 处理leading: 若不执行第一次, 且是首次触发, 更新lastTime为当前时间
18      if (!leading && lastTime === 0) {
19        lastTime = nowTime;
20      }
21
22      // 2. 计算距离下次可执行的时间
23      const remainingTime = interval - (nowTime - lastTime);
24
25      // 3. 满足执行条件: 要么剩余时间<=0 (超过间隔), 要么系统时间被篡改 (remainingTime
   异常大)
26      if (remainingTime <= 0 || remainingTime > interval) {
27        if (timer) {
```

```

28         clearTimeout(timer);
29         timer = null;
30     }
31     func.apply(context, args);
32     lastTime = nowTime;
33 } else if (trailing && !timer) {
34     // 4. 处理trailing: 延迟执行最后一次触发
35     timer = setTimeout(() => {
36         func.apply(context, args);
37         lastTime = Date.now();
38         timer = null;
39     }, remainingTime);
40 }
41 };
42 };
43
44 // 使用示例
45 const loadMore = throttle(() => {
46     console.log("滚动加载更多");
47 }, 1000);
48 window.addEventListener("scroll", loadMore);

```

ChatGPT优化点：

- 添加leading和trailing配置项，支持“是否执行第一次”“是否执行最后一次”，适配不同场景（如按钮防重复点击可设置leading=true, trailing=false）；
- 增加系统时间篡改判断（remainingTime > interval），避免因修改系统时间导致函数无法执行；
- 通过定时器处理trailing逻辑，确保最后一次触发能被执行；
- 兼容函数的this指向与参数传递，与防抖方法保持使用一致性。

5. 手机号/身份证号脱敏

功能描述：对敏感信息进行脱敏处理，如手机号中间4位替换为*，身份证号中间8位替换为*，保护用户隐私。

Code block

```

1  /**
2   * 敏感信息脱敏
3   * @param {string} value - 待脱敏的值（手机号/身份证号）
4   * @param {string} type - 类型（"phone": 手机号, "idCard": 身份证号）
5   * @returns {string} 脱敏后的字符串
6   */
7  export const desensitize = (value, type) => {
8      // 先校验值是否为字符串

```

```

9     if (getDataType(value) !== "string") {
10         console.warn("脱敏值必须为字符串类型");
11         return value;
12     }
13
14     // 类型映射：正则表达式与替换规则
15     const typeMap = {
16         phone: {
17             regex: /^(\d{3})\d{4}(\d{4})$/,
18             replace: "$1****$2",
19             tip: "手机号格式不正确"
20         },
21         idCard: {
22             regex: /^(\d{6})\d{8}(\d{4})$/,
23             replace: "$1*****$2",
24             tip: "身份证号格式不正确"
25         }
26     };
27
28     const config = typeMap[type];
29     if (!config) {
30         console.warn("不支持的脱敏类型，仅支持phone和idCard");
31         return value;
32     }
33
34     // 校验格式并脱敏
35     if (config.regex.test(value)) {
36         return value.replace(config.regex, config.replace);
37     } else {
38         console.warn(config.tip);
39         return value;
40     }
41 };
42
43 // 使用示例
44 desensitize("13812345678", "phone"); // "138****5678"
45 desensitize("110101199001011234", "idCard"); // "110101*****1234"

```

ChatGPT优化点：

- 采用类型映射对象管理正则与替换规则，便于后续扩展（如添加邮箱脱敏）；
- 增加参数类型校验与格式校验，通过warn提示错误，提升工具健壮性；
- 正则表达式精准匹配手机号（11位）与身份证号（18位），避免部分匹配导致的脱敏错误；
- 返回原始值而非抛出错误，防止因脱敏失败影响业务流程。

6. 数组去重（支持复杂类型）

功能描述：实现数组去重，支持基础类型、对象、数组等复杂类型，解决Set无法去重引用类型的问题。

Code block

```
1  /**
2   * 数组去重（支持基础类型与引用类型）
3   * @param {Array} arr - 待去重的数组
4   * @param {string} [key] - 引用类型去重的键名（如对象数组按id去重，传入"id"）
5   * @returns {Array} 去重后的新数组
6   */
7  export const uniqueArray = (arr, key) => {
8    // 先校验是否为数组
9    if (getDataType(arr) !== "array") {
10      console.warn("请传入数组类型数据");
11      return [];
12    }
13
14    // 空数组直接返回
15    if (arr.length === 0) return [];
16
17    // 1. 基础类型去重（利用Set的唯一性）
18    const firstType = getDataType(arr[0]);
19    const basicTypes = ["string", "number", "boolean", "undefined", "symbol",
20      "bigint", "null"];
21    if (basicTypes.includes(firstType) && !key) {
22      return [...new Set(arr)];
23    }
24
25    // 2. 引用类型去重（对象/数组）
26    const cache = new Map();
27    return arr.filter((item) => {
28      let cacheKey;
29      if (key) {
30        // 按指定键名去重（如对象数组按id）
31        if (getDataType(item) !== "object" || !item.hasOwnProperty(key)) {
32          console.warn(`数组元素不存在键名${key}，无法按该键去重`);
33          return true; // 保留无法处理的元素
34        }
35        cacheKey = item[key];
36      } else {
37        // 复杂类型按深拷贝后的字符串作为缓存键（避免引用不同但值相同的情况）
38        cacheKey = JSON.stringify(deepClone(item));
39      }
```



```

40     if (!cache.has(cacheKey)) {
41         cache.set(cacheKey, true);
42         return true;
43     }
44     return false;
45 });
46 };
47
48 // 使用示例
49 // 基础类型数组
50 uniqueArray([1, 2, 2, 3, 3, 3]); // [1,2,3]
51 // 对象数组按id去重
52 uniqueArray([{id:1,name:"a"}, {id:2,name:"b"}, {id:1,name:"a"}], "id"); //
    [{id:1,...}, {id:2,...}]
53 // 复杂数组去重
54 uniqueArray([[1,2], [3,4], [1,2]]); // [[1,2], [3,4]]

```

ChatGPT优化点：

- 区分基础类型与引用类型，基础类型用Set高效去重，引用类型用Map缓存处理，兼顾性能与功能；
- 添加key参数，支持对象数组按指定键名去重（开发中最常用场景），提升工具实用性；
- 引用类型去重时结合deepClone方法，避免因引用不同但值相同导致的重复保留问题；
- 增加错误提示，对不符合条件的元素保留而非过滤，减少业务风险。

7. 日期格式转换

功能描述：将日期对象、时间戳转换为指定格式的字符串（如"2024-05-20 13:14:52"），支持自定义格式占位符。

Code block

```

1  /**
2   * 日期格式转换
3   * @param {Date|number|string} date - 待转换的日期（日期对象/时间戳/日期字符串）
4   * @param {string} [format="YYYY-MM-DD HH:mm:ss"] - 目标格式（占位符：YYYY-年，MM-
    月，DD-日，HH-时，mm-分，ss-秒）
5   * @returns {string} 格式化后的日期字符串
6   */
7  export const formatDate = (date, format = "YYYY-MM-DD HH:mm:ss") => {
8      // 1. 处理日期输入：转换为Date对象
9      let targetDate;
10     const dateType = getDataType(date);
11     if (dateType === "date") {
12         targetDate = date;

```

```

13     } else if (dateType === "number") {
14         // 时间戳：支持秒级（10位）与毫秒级（13位）
15         targetDate = new Date(date.toString().length === 10 ? date * 1000 : date);
16     } else if (dateType === "string") {
17         targetDate = new Date(date);
18     } else {
19         console.warn("不支持的日期类型，请传入日期对象、时间戳或日期字符串");
20         return "";
21     }
22
23     // 2. 校验日期是否有效
24     if (isNaN(targetDate.getTime())) {
25         console.warn("无效的日期值");
26         return "";
27     }
28
29     // 3. 提取日期组件
30     const year = targetDate.getFullYear();
31     const month = String(targetDate.getMonth() + 1).padStart(2, "0"); // 月份从0开始，补0至2位
32     const day = String(targetDate.getDate()).padStart(2, "0");
33     const hour = String(targetDate.getHours()).padStart(2, "0");
34     const minute = String(targetDate.getMinutes()).padStart(2, "0");
35     const second = String(targetDate.getSeconds()).padStart(2, "0");
36
37     // 4. 替换格式占位符
38     const formatMap = {
39         YYYY: year,
40         MM: month,
41         DD: day,
42         HH: hour,
43         mm: minute,
44         ss: second
45     };
46
47     return format.replace(/YYYY|MM|DD|HH|mm|ss/g, (match) => formatMap[match]);
48 };
49
50 // 使用示例
51 formatDate(new Date()); // "2024-05-20 13:14:52"
52 formatDate(1716184492); // 秒级时间戳，转换为对应日期
53 formatDate("2024-05-20", "YYYY年MM月DD日"); // "2024年05月20日"

```

ChatGPT优化点：

- 支持多种日期输入类型（日期对象、秒级/毫秒级时间戳、日期字符串），提升兼容性；

- 通过padStart(2, "0")自动补0，确保月份、日期等组件为2位格式（如5月转为"05"）；
- 采用格式映射对象与正则替换，便于后续扩展更多占位符（如星期）；
- 增加日期有效性校验，避免因无效日期导致的NaN问题。

8. URL参数解析与拼接

功能描述：实现URL参数的解析（将query字符串转为对象）与拼接（将对象转为query字符串），支持数组参数与特殊字符处理。

Code block

```
1  /**
2   * URL参数解析：将query字符串转为对象
3   * @param {string} [url=window.location.href] - 待解析的URL（默认当前页面URL）
4   * @returns {Object} 解析后的参数对象
5   */
6  export const parseUrlParams = (url = window.location.href) => {
7    // 提取URL中的query部分（?后面的内容）
8    const queryStr = url.includes("?") ? url.split("?")[1] : "";
9    if (!queryStr) return {};
10
11    // 分割参数并解析
12    return queryStr.split("&").reduce((params, item) => {
13      const [key, value] = item.split("=");
14      if (!key) return params;
15
16      // 解码特殊字符（如中文、空格）
17      const decodeKey = decodeURIComponent(key);
18      const decodeValue = decodeURIComponent(value || "");
19
20      // 处理数组参数（如key[]=1&key[]=2）
21      if (decodeKey.endsWith("[]")) {
22        const arrKey = decodeKey.slice(0, -2);
23        params[arrKey] = params[arrKey] || [];
24        params[arrKey].push(decodeValue);
25      } else {
26        // 若同一key出现多次，转为数组
27        if (params.hasOwnProperty(decodeKey)) {
28          params[decodeKey] = [params[decodeKey], decodeValue];
29        } else {
30          params[decodeKey] = decodeValue;
31        }
32      }
33      return params;
34    }, {});
35  };
```

```

36
37 /**
38  * URL参数拼接：将对象转为query字符串
39  * @param {Object} params - 待拼接的参数对象
40  * @param {boolean} [addQuestionMark=true] - 是否在开头添加?
41  * @returns {string} 拼接后的query字符串
42  */
43 export const stringifyUrlParams = (params, addQuestionMark = true) => {
44   if (getDataType(params) !== "object" || Object.keys(params).length === 0) {
45     return addQuestionMark ? "?" : "";
46   }
47
48   // 处理参数并拼接
49   const queryArr = Object.entries(params).reduce((arr, [key, value]) => {
50     // 过滤undefined和null值
51     if (value === undefined || value === null) return arr;
52
53     // 处理数组参数（转为key[]=1&key[]=2格式）
54     if (getDataType(value) === "array") {
55       value.forEach((item) => {
56         arr.push(`${encodeURIComponent(key)}[]=${encodeURIComponent(item)}`);
57       });
58     } else {
59       arr.push(`${encodeURIComponent(key)}=${encodeURIComponent(value)}`);
60     }
61     return arr;
62   }, []);
63
64   const queryStr = queryArr.join("&");
65   return queryStr ? (addQuestionMark ? `?${queryStr}` : queryStr) : "";
66 };
67
68 // 使用示例
69 // 解析URL参数
70 parseUrlParams("https://example.com?name=张三
71 &age=20&hobby[]=game&hobby[]=music");
72 // { name: "张三", age: "20", hobby: ["game", "music"] }
73
74 // 拼接URL参数
75 stringifyUrlParams({ name: "张三", age: 20, hobby: ["game", "music"] });
76 // "?name=%E5%BC%A0%E4%B8%89&age=20&hobby[]=game&hobby[]=music"

```

ChatGPT优化点：

- 支持数组参数的解析与拼接（采用key[]格式），符合后端常见的数组参数接收规范；

- 通过encodeURIComponent/decodeURIComponent处理中文、空格等特殊字符，避免参数乱码；
- 解析时处理同一key多次出现的情况（转为数组），拼接时过滤undefined和null值；
- parseUrlParams默认解析当前页面URL，stringifyUrlParams支持控制是否添加开头的?，提升使用灵活性。

9. 图片转Base64

功能描述：将图片URL或本地文件转为Base64格式，常用于图片预览、本地存储等场景。

Code block

```
1  /**
2   * 图片转Base64格式
3   * @param {string} imgSrc - 图片地址（远程URL/本地文件路径/uni-app临时文件路径）
4   * @param {Object} [options={ quality: 0.92, type: "image/jpeg" }] - 配置项：
   *   quality（质量0-1）、type（图片类型）
5   * @returns {Promise}
```

（注：文档部分内容可能由 AI 生成）